UNITY PROGRAMMER CURRICULUM · LEVEL REFERENCE

From zero coding to mid-level

Seven levels, from never having written code to mid-level Unity programming: what each one teaches in C# and in Unity, the games that teach it, and the Unity certifications on the way.

LEVEL REFERENCE



Prepared by Moayad Rahhal · Game Development Technical Manager · September 2026, updated October 2026

The level ladder

Seven levels take a student from never having written code to mid-level Unity programming. Each level lists what to learn in C# and what to do in Unity, and the games chosen so that building them exercises that level's topics.

Level	Name	Starting point	Certification checkpoint
0	Zero	Never coded, never opened Unity	—
1	Beginner	Can follow along and change values	—
2	Builder	Writes simple scripts alone	—
3	Junior-ready	Builds small games alone, uses the docs	**Unity Certified User: Programmer**
4	Junior	Builds structured, multi-scene games	**Unity Certified Associate: Programmer**
5	Mid-level I	Writes clean, working game systems	—
6	Mid-level II	Designs architecture, ships production games	—

Objective tags. Topics carry the exam objective they prepare for: `U 2.1` is Certified User: Programmer objective 2.1 (Appendix A); `A·Programming`, `A·UI`, `A·Debugging` and `A·Assets` are the Certified Associate: Programmer exam domains (Appendix B).

Hours. Unity expects at least 150 hours of Unity use and training before the User exam, so Levels 0–3 together (class plus hands-on project time) should add up to about 150 hours.

Across all levels

- Debugging gets harder each level: Console → breakpoints → null references → Profiler → performance
- Code reading at every level: both exams test evaluating code, not only writing it
- Git from Level 4 onward, team workflow at Level 6
- Shipping at every level: each level ends with something built and published
- Portfolio: the Associate exam assumes a diverse range of Unity projects, so by the end of Level 4 each student should have 4–5 different shipped games

The games

Each level has a book for every game, and each book teaches the whole level, so a class can build any one of them. The **core** game is the one to build first, and the one a course with time for one game per level builds. From Level 5, only the core game teaches the whole level.

Level	Games	Status
0	Rocket Launch	Built
1	Catch the Falling Blocks	Built
2	Mini Golf (3D), Space Shooter (2D), Tank Arena (2D, top-down)	Built
3	**Knight Run** (2D platformer, core), Crypt Keys (2D dungeon crawler), Gate Guard (3D tower defence)	Built
4	**Arcane Duel** (card battler, core), Pocket Karts (3D kart racer), Juice Tycoon (idle tycoon)	Arcane Duel built; the others planned
5	**Lost Ruins** (3D action adventure, core), Box Pusher (2D puzzle with undo), Tiny Colony (RTS-lite, optional)	Planned
6	**Card Table** (online card game for mobile, core), Arena Online (real-time multiplayer, optional)	Planned

0

LEVEL

Zero

🎯 **Goal.** understand what a program is and make things happen in Unity.

Games: Rocket Launch.

C# · what to learn

- What a program is: instructions, order of execution, input → process → output
- Variables and basic types: `int`, `float`, `bool`, `string`
- Arithmetic, comparison and logic operators
- `if` / `else if` / `else`
- Methods: defining, calling, parameters, return values
- Comments: `//` and `/* */`, and writing comments that say what the code does `U 3.6`
- Reading errors: what a compile error looks like and what the message means

Unity · what to do

- Unity Hub, creating a project
- The editor windows and what each one is for: Scene, Game, Hierarchy, Inspector, Project, Console `U 4.1`
- Setting the external script editor (Preferences) `U 4.2`
- GameObjects, components, Transform (position, rotation, scale)
- Creating and attaching a script
- `Start()` and `Update()`: the game loop idea
- `Debug.Log` and the Console `U 1.1`

1 LEVEL Beginner

🎯 **Goal.** build a complete small game with input, spawning, collision, score and a build.

Games: Catch the Falling Blocks.

C# · what to learn

- Loops: `for`, `while`, `foreach`, `break`, `continue`
- Arrays `U 2.1`
- Scope: local vs. class-level variables
- Classes as blueprints, objects as instances (the concept only)
- Access modifiers: `public` vs. `private` `U 2.1`
- Logic and flow control: `&&`, `||`, `!`, ternary `?:`, `switch` `U 2.5`
- Enums
- String interpolation and formatting: given a log message, write the code that printed it `U 1.1`

Unity · what to do

- Keyboard and mouse input `U 2.4`
- `Time.deltaTime`, frame-independent movement
- `[SerializeField]` and tuning values in the Inspector
- Prefabs, `Instantiate`, `Destroy`
- 2D and 3D colliders, triggers vs. collisions
- Rigidbody basics: velocity vs. forces
- Tags and layers
- Simple UI: TextMeshPro text, buttons
- Audio: AudioSource, one-shot sounds
- Basic 2D sprites and 3D primitives
- Building to WebGL / PC `A·Programming`

LEVEL 2

Builder

🎯 **Goal.** write scripts that talk to each other, react to events and use the API confidently.

Games: Mini Golf (3D), Space Shooter (2D) and Tank Arena (2D, top-down). Each book teaches the whole level.

C# · what to learn

- Classes in practice: fields, properties, constructors
- Variable modifiers: `static`, `const`, `readonly`, `[SerializeField]` vs. `public` `U 2.1`
- `List<T>` and `Dictionary<K,V>` (intro): when to use each vs. an array `U 2.1`
- Function declarations: return types, parameters, overloading `U 2.2`
- Data types in depth: `int` vs. `float`, implicit and explicit conversion, type-mismatch errors `U 3.2`
- `null`: what it is, which object is null, and why `NullReferenceException` happens `U 1.2`

Unity · what to do

- Event functions: `Awake`, `Start`, `Update`, `FixedUpdate`, `LateUpdate`, `OnEnable`, `OnDisable`, and when each runs `U 3.1`
- `GetComponent`, and references between scripts and objects
- Coroutines and timers
- Vectors in practice: direction, distance, normalize
- Raycasts
- Touch input (and an intro to the Input System) `U 2.4` `U 3.1`
- UI change events: `onValueChanged` and `AddListener` on Slider, Toggle, Input Field, Dropdown `U 2.6`
- Particles for feedback
- Using the Unity docs to find the right class, method, property and arguments `U 1.3`
- Debugging: breakpoints, reading stack traces, finding the null object `U 1.2`

3 LEVEL Junior-ready

> 🎯 **Goal.** build complete games independently, with animated state machines and readable code.

Certification checkpoint: Unity Certified User: Programmer.

Games: Knight Run (2D platformer, the core game), Crypt Keys (2D dungeon crawler) and Gate Guard (3D tower defence).

C# · what to learn

- State machines in code: `enum` + `switch`
- Naming conventions: Unity and C# standards (PascalCase, camelCase, meaningful names) `U 3.5`
- Reading code and choosing the comment that describes it accurately `U 3.6`
- Evaluating code: "what does this print?", "find the error" (type errors, public/private mismatch) `U 3.2` `U 3.3`
- Recognizing class types: MonoBehaviour, ScriptableObject, plain C# class, ECS (recognition only, no DOTS) `U 3.4`

Unity · what to do

- Animator Controller: states, transitions, conditions, parameters (Trigger, Bool, Float, Int) `U 4.4`
- Building a functional state machine from given animation clips and property settings `U 4.3`
- Controlling the Animator from code: `SetTrigger`, `SetBool`, `SetFloat`, `SetInteger` `U 2.3`
- Animation Events, and the errors they cause when functions are declared wrongly `U 3.3`
- Enemy behavior as a state machine (in code and in the Animator)
- UI: health bars, score, menus
- Exam readiness: objective review, code-reading drills, official practice test

LEVEL 4

Junior

🎯 **Goal.** structured, multi-scene games with clean code, saving and version control.

Certification checkpoint: Unity Certified Associate: Programmer.

Games: Arcane Duel (card battler, the core game), Pocket Karts (3D kart racer) and Juice Tycoon (idle tycoon).

C# · what to learn

- Inheritance: base classes, `virtual` / `override`, `abstract` `A·Programming`
- Interfaces, and when to use an interface vs. inheritance `A·Programming`
- `Dictionary<K,V>` in depth, `Queue`, `Stack`, and choosing the right structure `A·Programming`
- Choosing data types deliberately `A·Programming`
- C# events, `Action`, delegates (intro)
- Static members: what they're for and the risks
- Coding standards: following a team style guide set by a lead, refactoring code to fit it `A·Programming` `A·Debugging`
- Debug messages that pinpoint why code fails (`Debug.LogWarning`, `Debug.LogError`), and finding the cause of a compile error in a given block of code `A·Debugging`
- Exceptions: `try` / `catch` basics

Unity · what to do

- Scene management: loading scenes, transitions, passing data between them `A·Programming`
- Saving: static variables, PlayerPrefs, JSON save files `A·Programming`
- UI in depth: Canvas scaling, anchors, pivots, layout groups `A·UI`
- Displaying data in UI elements, responding to input with UnityEvents `A·UI`
- ScriptableObjects for data
- Prefabs in depth: nested prefabs, prefab variants, overrides (apply / revert) and how a change propagates `A·Assets`
- Package Manager, Asset Store imports, resolving conflicts
- Version control: why it matters (history, rollback, collaboration); Git with Unity: `.meta` files, `.gitignore`, commits, branches `A·Assets`
- Object pooling

- Profiling tools: which tool for which problem (Profiler, Frame Debugger, Memory Profiler) `A·Debugging`
- New Input System
- Working inside an existing codebase: reading, evaluating and integrating code `A·Programming`
- Building to WebGL and PC `A·Programming`
- Exam readiness: domain review, code-evaluation drills, official practice test

LEVEL 5

Mid-level I

🎯 **Goal.** architecture and performance: systems that stay clean and fast as the game grows.

Games: Lost Ruins (3D action adventure, the core game, and the only one that teaches the whole level), Box Pusher (2D puzzle with undo) and Tiny Colony (RTS-lite, optional).

C# · what to learn

- Generics: writing your own generic classes and methods
- LINQ, and when not to use it in games (GC allocations)
- Delegates and events in depth, the observer pattern
- `async` / `await`, Tasks (and UniTask as the Unity-friendly option)
- Memory: value vs. reference types, structs, boxing, garbage collection
- SOLID principles at a practical level
- Design patterns used in games: Singleton (and its problems), Observer, Command, State, Factory, Object Pool, Service Locator
- Dependency injection (the concept, then a lightweight approach)

Unity · what to do

- Architecture: separating game logic from MonoBehaviours, event-driven systems
- Script execution order and the full MonoBehaviour lifecycle
- Asset loading: the `Resources` folder, and why to avoid it (measured: build size, start-up time, memory); then Addressables, which fix those costs
- Performance: Profiler deep dive, Frame Debugger, draw calls, batching, GC spikes
- Advanced Animator: blend trees, layers, avatar masks
- World building: Terrain, ProBuilder, LOD and occlusion culling; baked lighting and post-processing
- Cinemachine, Timeline

6

LEVEL

Mid-level II

🎯 **Goal.** production: testing, tooling, online features, mobile and team delivery.

Games: Card Table (online card game for mobile, the core game) and Arena Online (real-time multiplayer, optional).

C# · what to learn

- Unit testing and writing testable code
- HTTP / REST from C#, JSON serialization, error handling
- Cancellation and timeouts in async code
- Reading and reviewing other people's code

Unity · what to do

- Unity Test Framework: Edit Mode and Play Mode tests
- Editor scripting: custom Inspectors, tools for designers
- Mobile builds (Android / iOS) and platform-specific concerns
- Backend integration: Firebase / PlayFab (grouped by SDK)
- Multiplayer intro: Netcode for GameObjects concepts (client/server, sync)
- Team workflow: Git branching in a team, code reviews, build pipelines
- Shipping: store builds, versioning, crash reporting

APPENDIX

A Certified User: Programmer

Code	Objective (summary)	Level
1.1	Given a debug log message, write the code that produced it	0, 1
1.2	Given code and its error, find which object is null	2
1.3	Pick the right API class member and syntax for a task	2
2.1	Variables, modifiers, arrays, Lists and Dictionaries	1, 2
2.2	Construct a valid function declaration	2
2.3	Choose the function that triggers a state (incl. Animator)	3
2.4	Build an input listener (keyboard and touch)	1, 2
2.5	Logic and flow control operators	1
2.6	Respond when a UI element reports a change	2
3.1	Manage event functions (incl. keyboard and touch input)	2
3.2	Spot a wrongly declared data type	2, 3
3.3	Spot public/private misuse (incl. Animation Events)	3
3.4	Tell an ECS class from other class types	3
3.5	Recognize Unity naming conventions	3
3.6	Pick the comment that accurately describes code	0, 3
4.1	Purpose and features of the editor windows	0
4.2	Change the default scripting IDE	0
4.3	Build a state machine from clips and property settings	3
4.4	Program a state machine in the Animator Controller	3

APPENDIX

B Certified Associate: Programmer

Domain	Objective (summary)	Level
Programming	Evaluate code for integration into a system architected by a lead	4
Programming	Apply coding standards set by senior programmers	3, 4
Programming	Determine code for a specified interaction or logic	1–4
Programming	Implement transitions between scenes	4
Programming	Save data between scenes and sessions (static, PlayerPrefs)	4
Programming	Use Unity API methods, given the API docs	2
Programming	Choose GameObject properties, scripts and components for a task	1, 2
Programming	Inheritance vs. interfaces	4
Programming	Choose data structures: lists, arrays, dictionaries	2, 4
Programming	Choose data types: floats, bools, strings	2, 4
Programming	Build to WebGL or PC	1, 4
UI	Lay out UI with anchors, pivots and groups	4
UI	Display data in UI elements	1, 4
UI	Respond to user input with the UnityEvent system	2, 4
Debugging	Use debug messages to find why code fails	0–4
Debugging	Identify the cause of a compile error	0, 3, 4
Debugging	Identify null-variable errors	2
Debugging	Refactor code to fit coding standards	4
Debugging	Select profiling tools for performance problems	4
Assets	Use prefabs in a scene	1
Assets	Change nested prefabs and prefab variants, and predict the outcome	4
Assets	Primary purposes of version control	4

Prerequisites Unity lists for this exam (all covered by Levels 1–4): 2–3 semesters of Unity classwork or equivalent; a diverse range of Unity projects; importing assets or code from the Asset Store or Package Manager and resolving conflicts; debugging non-complex problems; interpreting, integrating and modifying well-documented existing code; basic scene management; creating and using prefabs; deploying a basic build.